Low Level Design

For

Deployment of Education App to Kubernetes

Course Name: **DevOps**

BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE & ENGINEERING

BY

Sr.no	Student Name	Enrolment Number
1	ABHIKET THAKUR	EN22CS301030
2	ADITYA PALIWAL	EN22CS301055
3	AARUSHI MITTAL	EN24CS3T10026
4	AKASH KUSHWAHA	EN22CS301083
5	AKSHAY DASHORE	EN22CS301096
6	AKSHITA RATHORE	EN22CS301100

Institution Name: Medi-caps University – Datagami Skill Based Course

Group: 04D1

Project No: DO-04

Industry Mentor Name: Aashruti Shah

University Mentor Name: Dr. Ritesh Joshi

Department of Computer Science & Engineering

Faculty of Engineering

MEDICAPS UNIVERSITY, INDORE- 453331

Jan - May 2026

1. Document Overview

1.1 Scope of the document

This document outlines the design, development, and deployment of a containerized Education web application using Kubernetes. It covers the complete workflow starting from application containerization using Docker to orchestration and deployment on a Kubernetes cluster (Minikube). The scope includes defining Kubernetes Deployment and Service configurations to manage application pods, ensure load balancing, and maintain high availability. It also explains the implementation of zero-downtime rolling updates, which allows seamless upgrades of the application without interrupting user access.

Additionally, the document highlights the integration of CI/CD pipelines using tools such as GitHub Actions or Jenkins to automate the build, testing, and deployment process. The focus is on demonstrating real-world DevOps practices including version control, container management, and automated deployment. However, this document is limited to a local Kubernetes environment using Minikube and does not cover production-grade cloud deployment in detail. Advanced features like large-scale distributed systems, enterprise security configurations, and multi-cluster management are beyond the scope of this project.

1.2 Intended Audience

The primary audience includes:

- Project Evaluators and Academic Reviewers
- Development Team Members
- Future Maintainers and Enhancers
- Research-Oriented Audience

1.3 System overview

The system is designed to deploy and manage a containerized Education web application using Kubernetes. It follows a modern DevOps architecture that integrates containerization, orchestration, and continuous deployment to ensure scalability, reliability, and high availability. The core of the system is a React-based Education application that is packaged into a Docker container. This containerized application is then deployed onto a Kubernetes cluster using Minikube, which acts as a local testing environment for orchestration.

Kubernetes manages the application through its key components such as Deployments and Services. The Deployment ensures that a specified number of application instances (pods) are always running, providing fault tolerance and self-healing capabilities. The Service component exposes the application to users and distributes incoming traffic evenly across all running pods, ensuring efficient load balancing.

A significant feature of the system is the implementation of zero-downtime rolling updates. Whenever a new version of the application is released, Kubernetes gradually replaces old pods with new ones without interrupting the availability of the application. This ensures a seamless user experience even during updates. The system also incorporates a CI/CD pipeline using tools like GitHub Actions or Jenkins. This pipeline automates the process of building the Docker image, pushing it to a container registry, and deploying it to the Kubernetes cluster. This reduces manual intervention and improves deployment efficiency.

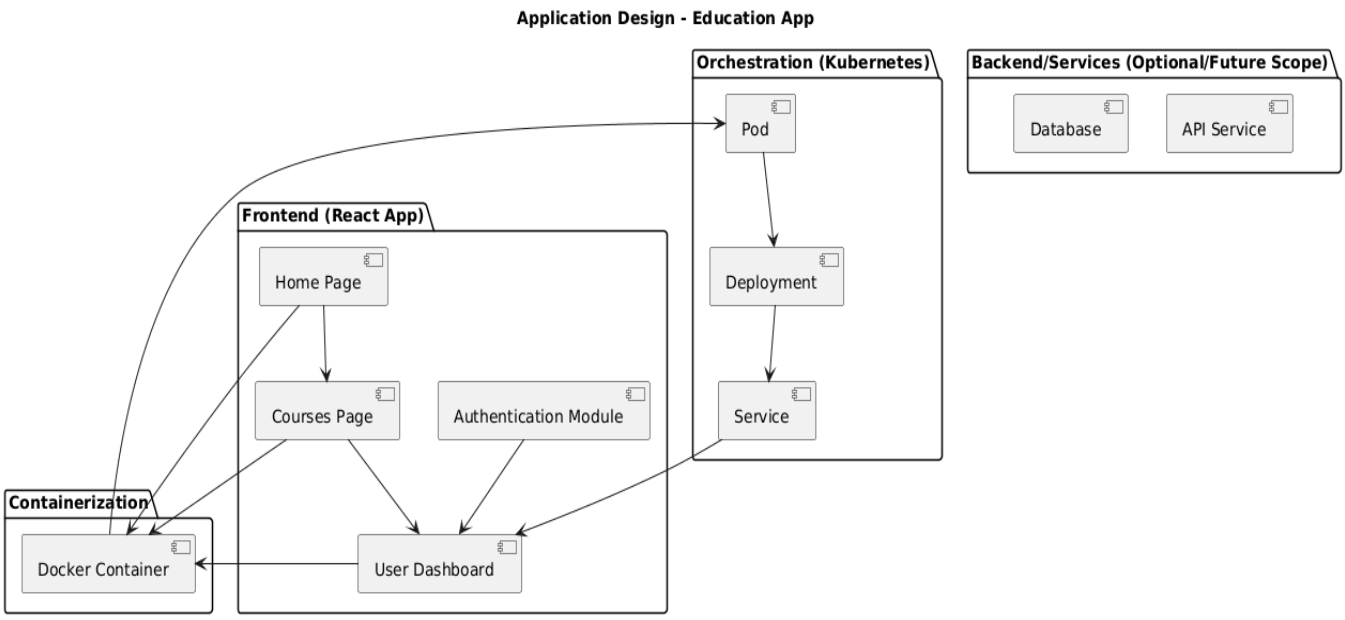
Overall, the system demonstrates how modern tools and technologies can be combined to create a scalable, automated, and resilient application deployment environment.

2. System Design

In this project, the system design focuses on deploying a containerized Education application using Kubernetes. It involves designing how different components—such as the React frontend, Docker containers, Kubernetes deployments, and services—interact with each other. The design ensures high availability, fault tolerance, and smooth application updates through features like rolling deployments. By incorporating modern DevOps practices, the system design also includes automation through CI/CD pipelines, enabling continuous integration and seamless delivery of application updates.

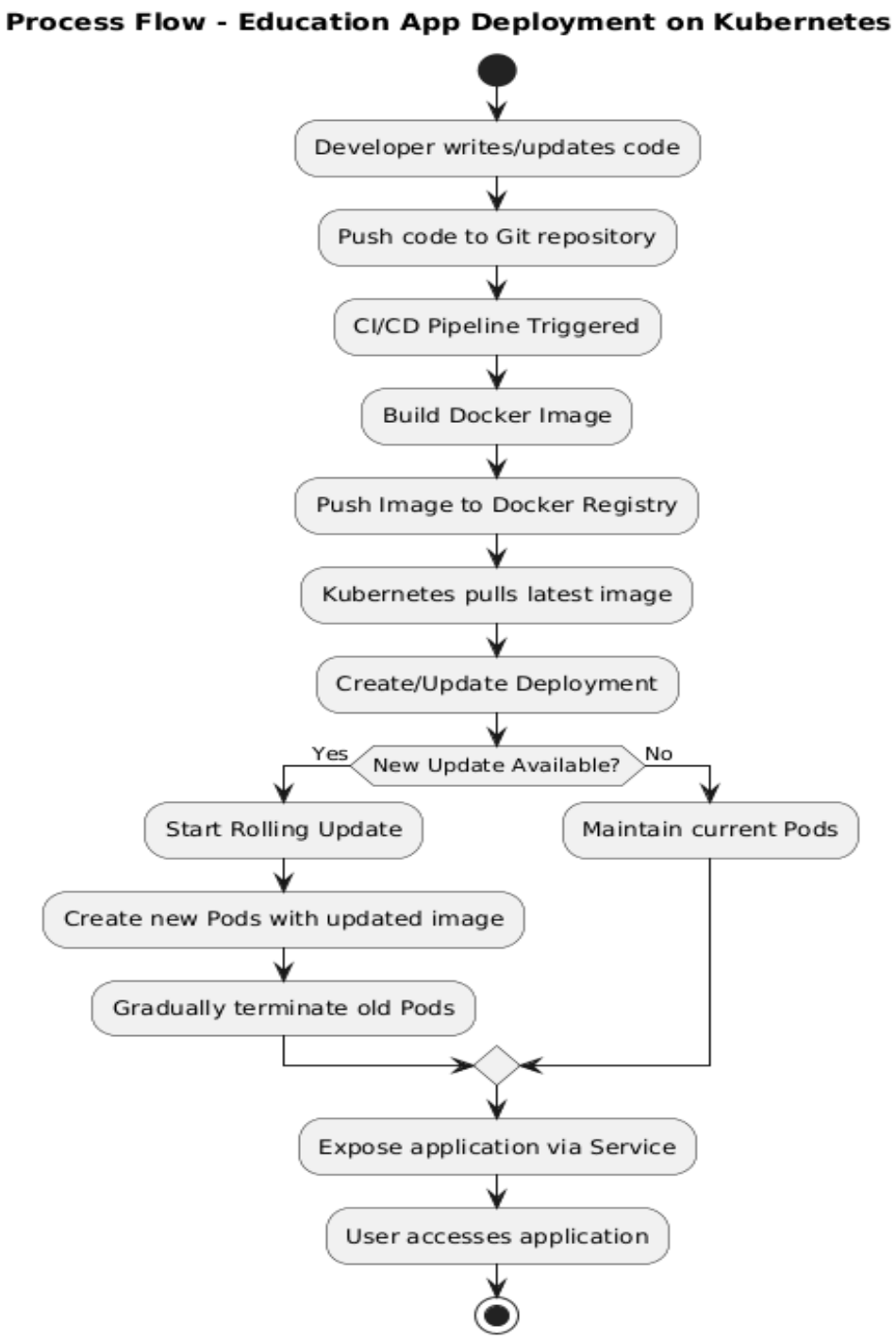
2.1 Application Design

Application design focuses on the structure and behavior of the application, including how its components are organized and interact with each other. It defines the user interface, functionality, and internal logic required to meet user needs.



In this project, the application design involves building a React-based Education app with a modular and responsive interface. The design ensures smooth user interaction, efficient data handling, and compatibility with containerization and deployment processes. It also considers performance, scalability, and ease of integration with Kubernetes-based deployment.

2.2 Process Flow



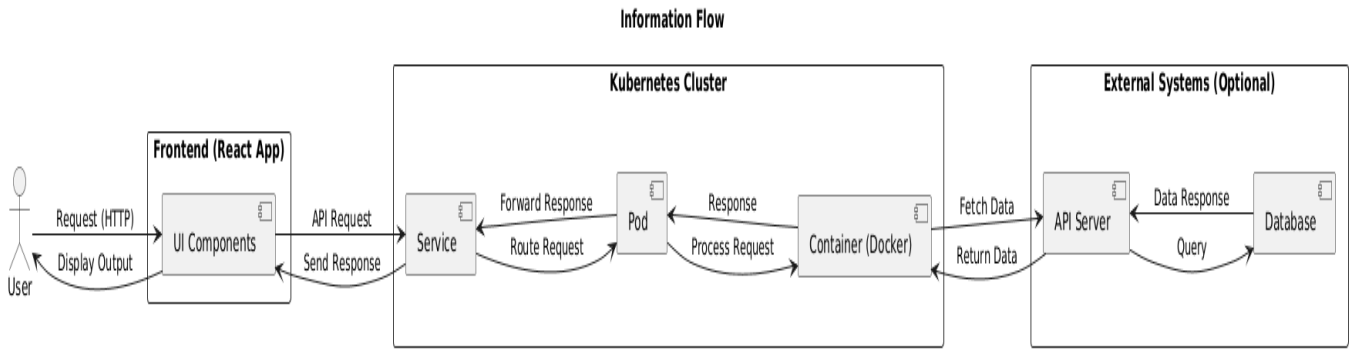
The process flow describes the sequence of steps involved in developing, containerizing, and deploying the Education application onto a Kubernetes cluster. It illustrates how the system moves from code development to continuous integration, container creation, and automated deployment.

The flow also highlights the role of CI/CD pipelines in streamlining the build and deployment process, as well as Kubernetes in managing application updates through rolling deployments. This ensures that the application remains available to users while updates are applied seamlessly.

2.3 Information Flow

The information flow represents how data moves within the system between different components during operation. It describes the exchange of information from the user to the application, through the deployed services, and back to the user.

In this project, information flows from the user interface of the React application to the containerized environment managed by Kubernetes, where requests are handled by pods and routed through services. This structured flow ensures efficient communication, proper request handling, and a seamless user experience.

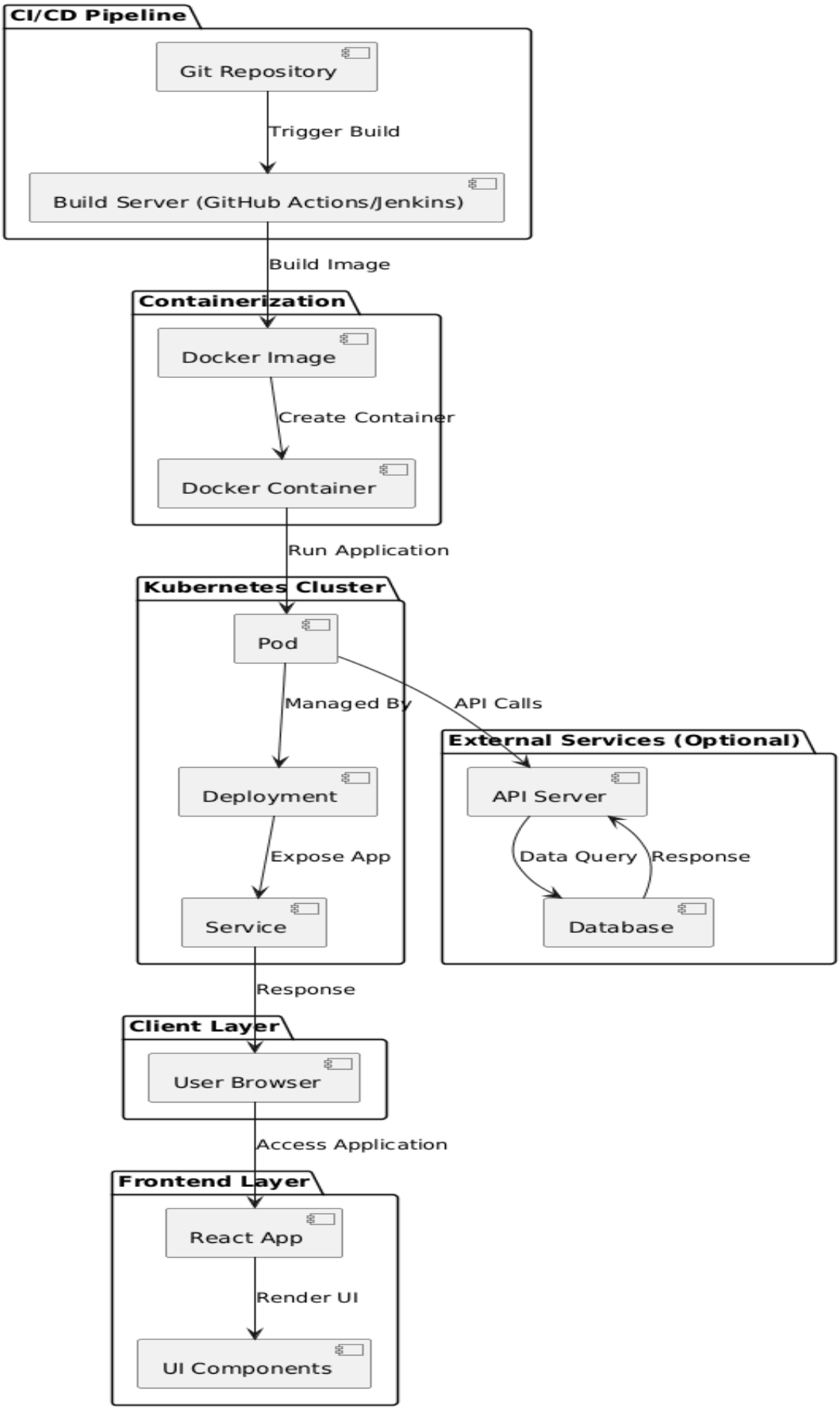


2.4 Components Design

Component design defines the individual modules of the system and their interactions to achieve the overall functionality. It focuses on breaking down the system into manageable and reusable components, each responsible for a specific task.

In this project, the component design includes key elements such as the React frontend, Docker containers, CI/CD pipeline, and Kubernetes resources like Deployments, Pods, and Services. These components work together to ensure efficient application deployment, scalability, and smooth communication within the system.

Component Design - Education App on Kubernetes



2.5 Key Design Considerations

1. Scalability: The system is designed to handle increasing user traffic by scaling the number of pods in the Kubernetes cluster. This ensures that the application can perform efficiently under varying loads.

2. High Availability: Multiple replicas of the application are maintained using Kubernetes Deployments to ensure that the system remains available even if some pods fail.

3. Zero-Downtime Deployment: Rolling updates are implemented to allow seamless updates of the application without interrupting user access, ensuring continuous availability.

4. Fault Tolerance: Kubernetes automatically detects and replaces failed pods, ensuring system stability and minimizing downtime.

5. Containerization: Using Docker ensures consistency across different environments by packaging the application along with its dependencies.

6. Automation (CI/CD): Integration with CI/CD tools like GitHub Actions or Jenkins automates the build and deployment process, reducing manual errors and improving efficiency.

7. Load Balancing: Kubernetes Services distribute incoming traffic evenly across multiple pods, ensuring optimal resource utilization and performance.

8. Maintainability: The use of modular design and configuration files (YAML) makes the system easy to manage, update, and extend.

9. Portability: The containerized application can be deployed across different environments (local, cloud, hybrid) without major changes.

10. Security (Basic Level): Basic security practices such as controlled access, secure image usage, and environment isolation are considered in the design

2.6 API Catalogue

1. User Authentication API

- Endpoint: /api/auth/login
- Method: POST
- Description: Authenticates the user and returns an access token

2. Get Courses API

- Endpoint: /api/courses
- Method: GET

- Description: Retrieves the list of available courses

3. Get Course Details API

- Endpoint: /api/courses/{id}
- Method: GET
- Description: Fetches detailed information about a specific course

4. User Registration API

- Endpoint: /api/auth/register
- Method: POST
- Description: Registers a new user

5. Enroll in Course API

- Endpoint: /api/enroll
- Method: POST
- Description: Enrolls a user in a selected course

6. Health Check API

- Endpoint: /api/health
- Method: GET
- Description: Checks if the application is running properly (used by Kubernetes for monitoring)

4. References

- Kubernetes Documentation. *Kubernetes Official Website*. Available at: https://kubernetes.io/docs/
- Docker Documentation. *Docker Official Website*. Available at: https://docs.docker.com/
- React Documentation. *React Official Website*. Available at: https://react.dev/
- GitHub Actions Documentation. *GitHub Docs*. Available at: https://docs.github.com/en/actions
- Jenkins Documentation. *Jenkins Official Website*. Available at: https://www.jenkins.io/doc/
- Nginx Documentation (for serving React apps). Available at: https://nginx.org/en/docs/
- Red Hat. *Introduction to Kubernetes*. Available at: https://www.redhat.com/en/topics/containers/what-is-kubernetes
- Google Cloud. *Kubernetes Engine Concepts*. Available at: https://cloud.google.com/kubernetes-engine/docs/concepts
- Burns, Brendan, et al. *Kubernetes: Up and Running*. O'Reilly Media, 2019.
- Merkel, Dirk. *Docker: Lightweight Linux Containers for Consistent Development and Deployment*. Linux Journal, 2014.